

Awarding Body:

Arden University, Berlin

Programme Name:

MSc in Data Science

Module Name:

COM7019 - Artificial Intelligence and Neural Networks

Assessment Title:

Artificial Intelligence and Neural Network Portfolio

Student number:

25199053

Tutor Name:

Ali Vaisifard

Word Count:

3303

Table of Contents

1. Introduction.....	3
2. Data description and exploratory analysis.....	3
3. Methodology.....	6
3.1 Forecasting task.....	6
3.2 Chronological split.....	6
3.3 Windowing and normalisation.....	7
3.4 Baseline and fairness controls.....	8
4. Neural network architectures.....	8
4.1 LSTM and GRU.....	8
4.2 CNN-LSTM hybrid extension.....	9
5. Experimental programme and optimisation.....	10
5.1 Depth comparison.....	10
5.2 Dropout ablation.....	11
5.3 Window-horizon ablation.....	12
5.4 Learning-rate experiment.....	12
5.5 Hybrid model and fine-tuning.....	13
6. Results and comparative analysis.....	14
6.1 Overall ranking.....	14
6.2 LSTM versus GRU.....	17
6.3 What the prediction plot shows.....	18
6.4 Summary of Task 1 findings.....	18
7. Critical evaluation.....	19
7.1 Interpreting performance.....	19
7.2 Why the ceiling is low.....	19
7.3 Business and investment implications.....	19
7.4 Reliability and limitations of the study.....	20
7.5 Ethical implications.....	20
7.6 Position relative to developments in the field.....	20
8. Conclusion and reflection.....	21
Reference List.....	22
Appendix A. Code repository and final artefacts.....	23
A.1 Primary executable notebook.....	23
A.2 Authorised dataset.....	23
A.3 Results tables (frozen run).....	24

1. Introduction

This portfolio responds to a brief in which I act as an Artificial Intelligence Analyst at a financial technology company. My line manager has commissioned a deep learning investigation into whether historical stock price data can support short-term trend prediction, so that the business can judge whether a neural network approach is worth pursuing further.

The brief requires design, implementation and comparison of LSTM and GRU on one authorised time-series dataset, plus configuration experiments and generalisation techniques (dropout, early stopping and fine-tuning). I also add a CNN-LSTM hybrid under the same protocol to test whether a convolutional front-end helps one-step-ahead forecasting.

The rest of the report covers data, method, experiments, results, critical evaluation and a short reflection.

2. Data description and exploratory analysis

The authorised file `'Stock_Price_Data_[3921].csv'` contains 9,909 trading days from 12 December 1980 to 1 April 2020. Each row records Date, Open, High, Low, Close, Adjusted Close and Volume for a single equity. Quality checks confirmed that there are no missing values, no duplicate dates, and that rows are already in chronological order (Figures 1 and 2).

Number of rows: 9909

Columns: ['Date', 'Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume']

	Date	Open	High	Low	Close	Adj Close	Volume
0	1980-12-12	0.513393	0.515625	0.513393	0.513393	0.406782	117258400
1	1980-12-15	0.488839	0.488839	0.486607	0.486607	0.385558	43971200
2	1980-12-16	0.453125	0.453125	0.450893	0.450893	0.357260	26432000
3	1980-12-17	0.462054	0.464286	0.462054	0.462054	0.366103	21610400
4	1980-12-18	0.475446	0.477679	0.475446	0.475446	0.376715	18362400

Figure 1. Dataset load preview in Colab

```

Date range: 1980-12-12 to 2020-04-01

Missing values per column:
Date          0
Open          0
High          0
Low           0
Close         0
Adj Close     0
Volume        0
dtype: int64

Duplicate dates: 0
Dates already sorted oldest-to-newest: True

Min price: 0.1556382328271865 | Max price: 327.20001220703125

Daily return mean: 0.001065
Daily return std : 0.0287
Lag-1 autocorrelation: 0.0189

```

Figure 2. Data quality checks in Colab

I predict Adjusted Close rather than Close because it corrects for dividends and better reflects investor returns; unadjusted Close can contain administrative jumps. Across four decades the level grows from under 0.16 USD to over 327 USD, which matters for scaling.

Such series are hard to forecast. Daily equity returns behave close to a random walk: lag-1 autocorrelation is near zero, volatility clusters after shocks, and prices already embed news and macro information that a univariate model never sees (Fama, 1970). Persistence (tomorrow equals today) is therefore a strong benchmark. I treat this as a data property, not weak modelling, and return to it when interpreting performance.

Figures 3 to 5 show recent price action and the March 2020 crash. They do not prove predictability; they show that the test period includes a real stress regime.



Figure 3. Candlestick chart of the last 60 trading days

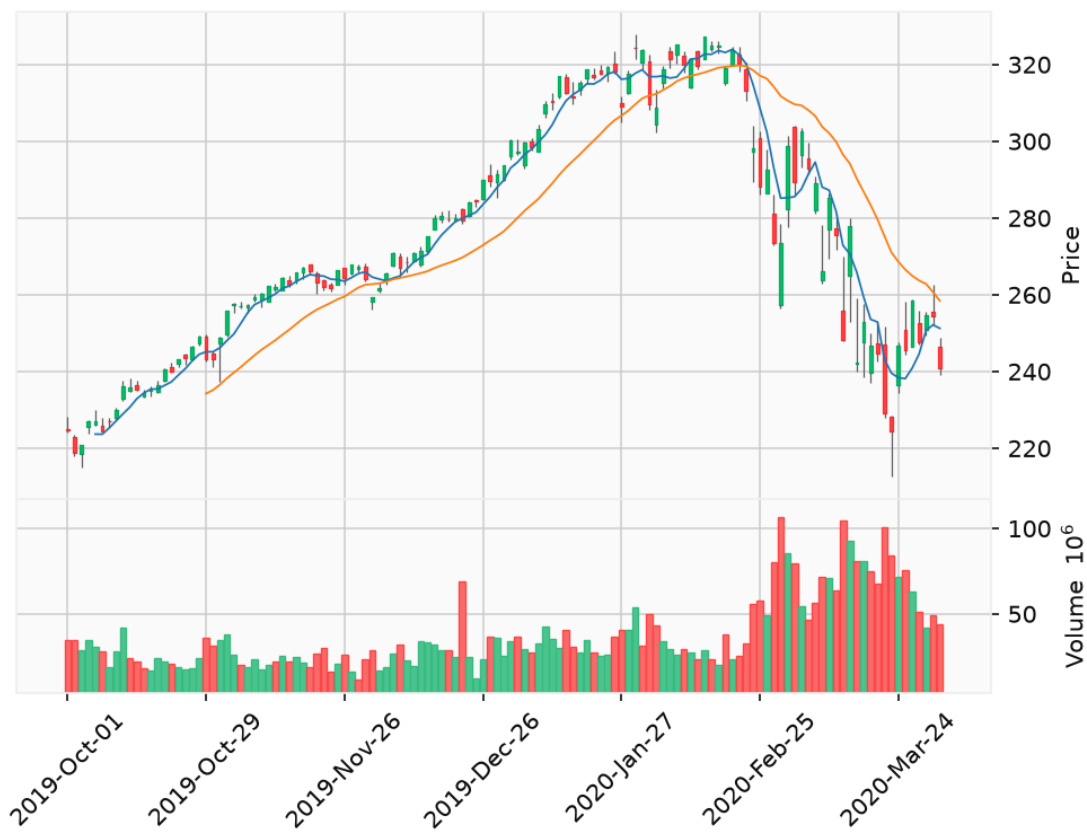


Figure 4. Candlestick chart from October 2019 to April 2020



Figure 5. Candlestick zoom on the February-March 2020 crash

3. Methodology

3.1 Forecasting task

The modelling task is one-step-ahead regression: given a window of past Adjusted Close values, predict the next day's Adjusted Close. All neural models are trained with mean squared error (MSE) loss and Adam optimisation. Performance is reported in USD using root mean squared error (RMSE), mean absolute error (MAE) and mean absolute percentage error (MAPE).

3.2 Chronological split

The sample is split by time, not at random: shuffling would leak future days into training and inflate accuracy. I use a chronological 70/15/15 split (train to June 2008, validation to May 2014, test to April 2020). Selection uses validation RMSE only; the test set is scored once per configuration. Figure 6 marks the three periods.



Figure 6. Adjusted Close with train, validation and test periods

3.3 Windowing and normalisation

Models receive sliding windows of past trading days. Following common practice in finance machine learning, lookbacks are defined in trading days rather than calendar days: one week (1W) equals 5 days, one month (1M) equals 21 days, and one year (1Y) equals 252 days. Primary architecture comparisons use the 1M window. A later ablation repeats one GRU configuration across all three horizons. Scaling needs care. A MinMax scaler fitted only on training data would map almost every test price outside the fitted range because of multi-decade growth. Fitting on the full series would leak future levels into training. Instead, each window is normalised against its own last price, so every input becomes a relative change. The target is expressed in the same relative units and converted back to USD before scoring (Figures 7 and 8).

```
... Total samples: 9888
X shape: (9888, 21, 1) (samples, timesteps, features)
Last value of first window (should be 0.0): 0.0
```

Figure 7. Sliding-window shapes and per-window normalisation check

```
... Train samples: 6915
   Val samples  : 1486
   Test samples : 1487
```

Figure 8. Train, validation and test sample counts after windowing

This keeps inputs comparable across eras without leaking the train/test boundary (Chollet and Watson, 2025).

3.4 Baseline and fairness controls

Before any neural network is trained, I compute a persistence baseline on the test set: the prediction for day $t+1$ is the observed price on day t . On this dataset the baseline scores test RMSE 3.2380, MAE 1.8269 and MAPE 1.159% (Figure 9). Every subsequent model is judged against those numbers.

```
Baseline (persistence) on TEST set:  
RMSE : 3.238  
MAE  : 1.8269  
MAPE : 1.159 %
```

Figure 9. Persistence baseline metrics on the test set

Fair comparison is enforced by a shared protocol: seed 25199053, identical chronological split, identical primary window for architecture comparisons, MSE loss, Adam with learning rate 0.001 unless an experiment states otherwise, batch size 64, a maximum of 40 epochs, and early stopping with patience 5 that restores the best validation weights. Model names encode architecture, depth, dropout, window and learning-rate variants so that tables remain readable without opening the code.

4. Neural network architectures

4.1 LSTM and GRU

LSTM and GRU are the two architectures required by the brief. Both are recurrent models designed to retain information across a sequence while mitigating vanishing-gradient problems that affect simple recurrent networks (Chollet and Watson, 2025; Ekman, 2021). In Keras, LSTM and GRU cells use the standard gated activations: sigmoid gates for update/forget-style control and tanh for candidate states. The final dense layer is linear because the task is unbounded regression in relative-price space.

I compare two depths for each family under dropout 0.2 after recurrent layers: a single 64-unit layer, and a stacked 64-then-32 design. The single-layer GRU has 12,929 trainable parameters; the single-layer LSTM has 16,961. The two-layer GRU and LSTM have 22,305 and 29,345 parameters respectively. Figures 10 to 13 summarise these layouts.

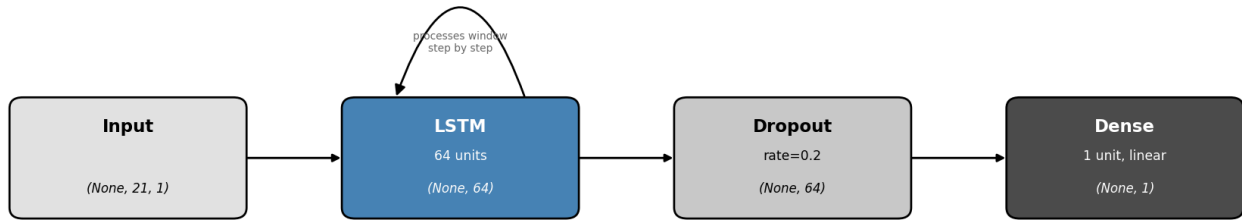


Figure 10. One-layer LSTM architecture (64 units, dropout 0.2)

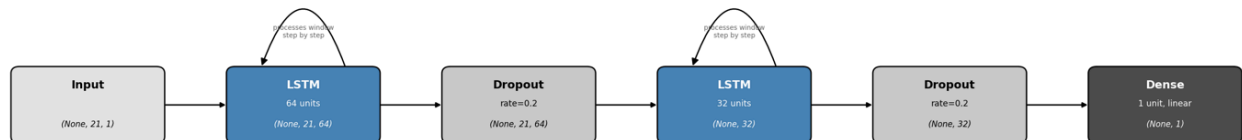


Figure 11. Two-layer LSTM architecture (64-32 units, dropout 0.2)

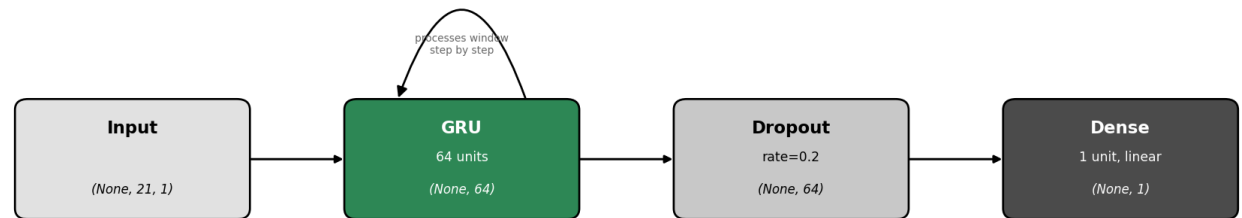


Figure 12. One-layer GRU architecture (64 units, dropout 0.2)

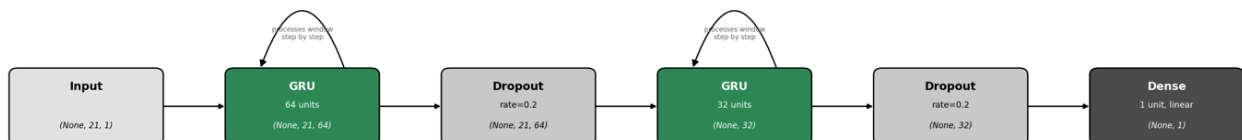


Figure 13. Two-layer GRU architecture (64-32 units, dropout 0.2)

4.2 CNN-LSTM hybrid extension

As an extension beyond the brief's minimum requirement, I add a hybrid that places a one-dimensional convolution in front of an LSTM. The convolutional stage uses 32 filters, kernel size 3 and ReLU activation, followed by max-pooling, then a 64-unit LSTM with dropout 0.2 and a linear output. The intention is that local short patterns are extracted before the recurrent layer models longer dependence within the window. This design choice is motivated by recent applied work in which convolution-plus-recurrence hybrids are reported to improve stock forecasting relative to standalone recurrent baselines (Bhanujyothi and Jacob, 2025; Zhao et al., 2025; Rokde et al., 2025), although those gains are context-dependent and must be tested under the same protocol used here. Figure 14 shows the hybrid layout (25,025 parameters).

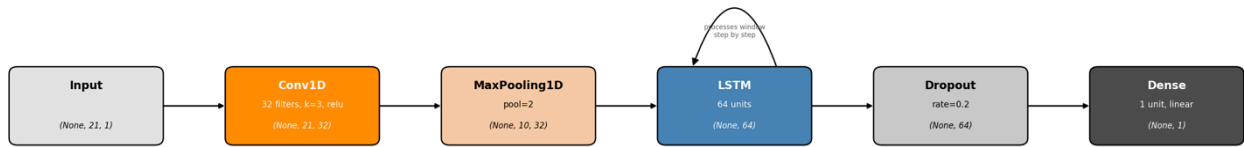


Figure 14. CNN-LSTM hybrid architecture

5. Experimental programme and optimisation

Experiments ran in stages. Validation RMSE governed selection so the test set stayed untouched until final scoring. Dropout and early stopping with best-weight restoration controlled overfitting (Chollet and Watson, 2025; Ekman, 2021).

5.1 Depth comparison

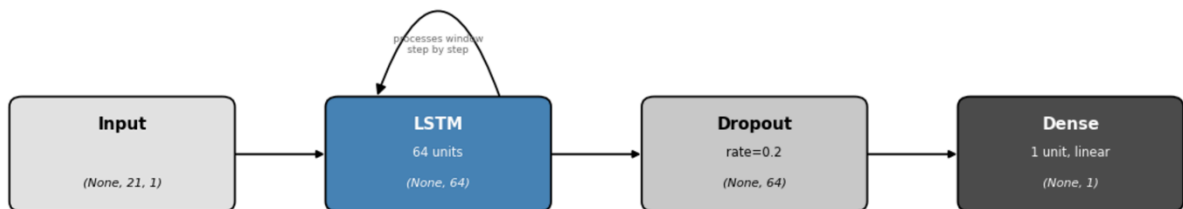
Four base models were trained on the 1M window: LSTM-64, LSTM-64-32, GRU-64 and GRU-64-32, all with dropout 0.2 (Figure 15). On validation RMSE the deeper LSTM was narrowly best among LSTMs (0.8741 versus 0.8742), while the single-layer GRU was best among GRUs (0.8735 versus 0.8738) (Figure 16). The gaps are under 0.1%, which is itself a finding: on this series, adding a second recurrent layer buys almost nothing.

Model: "LSTM_1Layer_64Units_Dropout0p2_Win1Month_21d"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 64)	16,896
dropout (Dropout)	(None, 64)	0
dense (Dense)	(None, 1)	65

Total params: 16,961 (66.25 KB)
 Trainable params: 16,961 (66.25 KB)
 Non-trainable params: 0 (0.00 B)

Figure 5: LSTM_1Layer_64Units_Dropout0p2_Win1Month_21d



--- Training: LSTM_1Layer_64Units_Dropout0p2_Win1Month_21d ---
 Finished in 9.6 seconds, 6 epochs
 LSTM_1Layer_64Units_Dropout0p2_Win1Month_21d -> Test RMSE: 3.2358 | MAE: 1.8255
 3.235765582344033

Figure 15. Training log for the one-layer LSTM

```

... Validation RMSE for LSTM models:
    LSTM_1Layer_64Units_Dropout0p2_Win1Month_21d : 0.8742
    LSTM_2Layer_64-32Units_Dropout0p2_Win1Month_21d : 0.8741
    Best LSTM : LSTM_2Layer_64-32Units_Dropout0p2_Win1Month_21d

Validation RMSE for GRU models:
    GRU_1Layer_64Units_Dropout0p2_Win1Month_21d : 0.8735
    GRU_2Layer_64-32Units_Dropout0p2_Win1Month_21d : 0.8738
    Best GRU : GRU_1Layer_64Units_Dropout0p2_Win1Month_21d

```

Figure 16. Validation RMSE ranking of LSTM and GRU models

Corresponding test RMSE values were 3.2358 (LSTM-1L), 3.2346 (LSTM-2L), 3.2383 (GRU-1L) and 3.2367 (GRU-2L). Against the persistence baseline of 3.2380, the two LSTM configurations and the deeper GRU sit slightly ahead, while the single-layer GRU with dropout is essentially level with persistence.

5.2 Dropout ablation

Each stage-one winner was retrained without dropout. For the LSTM, removing dropout worsened test RMSE from 3.2346 to 3.2387, so dropout helped generalisation. For the GRU, the no-dropout run scored 3.2381 against 3.2383 with dropout, a practical tie (Figures 17 and 18). I therefore treat dropout at 0.2 as a safe default rather than a universal win for every architecture.

```

--- Training: LSTM_2Layer_64-32Units_NoDropout_Win1Month_21d ---
Finished in 7.3 seconds, 6 epochs
LSTM_2Layer_64-32Units_NoDropout_Win1Month_21d → Test RMSE: 3.2387 | MAE: 1.8267
3.238664171413619

```

Figure 17. Dropout ablation result for the selected LSTM

```

--- Training: GRU_1Layer_64Units_NoDropout_Win1Month_21d ---
Finished in 7.6 seconds, 7 epochs
GRU_1Layer_64Units_NoDropout_Win1Month_21d → Test RMSE: 3.2381 | MAE: 1.8249
3.238112679585892

```

Figure 18. Dropout ablation result for the selected GRU

5.3 Window-horizon ablation

Using the no-dropout single-layer GRU, I compared trading-day horizons 1W, 1M and 1Y. The 1M window remained best (test RMSE 3.2381). The 1W window was worse (3.2554) and the 1Y window was worst (3.2762) despite 45.7 seconds of training. Shorter windows appear to lose useful context; the year-long window adds noise and compute without payoff for one-step-ahead level prediction. Figure 19 summarises the ablation.

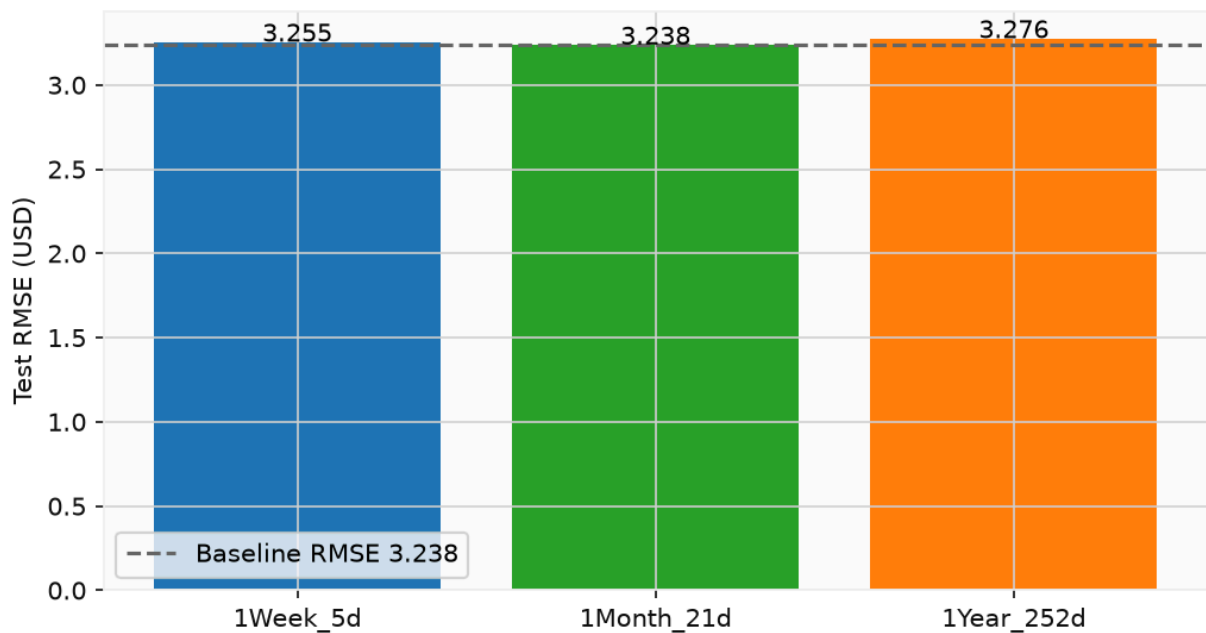


Figure 19. Test RMSE by lookback window (1W, 1M, 1Y)

5.4 Learning-rate experiment

Lowering the learning rate from 0.001 to 0.0001 on the single-layer GRU produced the strongest recurrent-only result: test RMSE 3.2339 and MAE 1.8245, in only a few epochs before early stopping (Figure 20). That run ranks second overall and clearly beats persistence.

```
...
--- Training: GRU_1Layer_64Units_NoDropout_LR0p0001_Win1Month_21d ---
Finished in 6.6 seconds, 6 epochs
GRU_1Layer_64Units_NoDropout_LR0p0001_Win1Month_21d → Test RMSE: 3.2339 | MAE: 1.8245
3.233908378551294
```

Figure 20. GRU training result with learning rate 0.0001

5.5 Hybrid model and fine-tuning

Under the identical 1M protocol, the CNN-LSTM hybrid reached test RMSE 3.2277, the best result in the investigation and about 0.0103 USD (roughly 0.32%) better than persistence (Figure 21). Fine-tuning at one-tenth the learning rate did not improve on that base: the fine-tuned hybrid scored 3.2361, worse than 3.2277 (Figure 22). The negative fine-tuning result is reported because the brief asks for fine-tuning as a generalisation technique; honesty about a failed improvement is part of a correct experimental record.

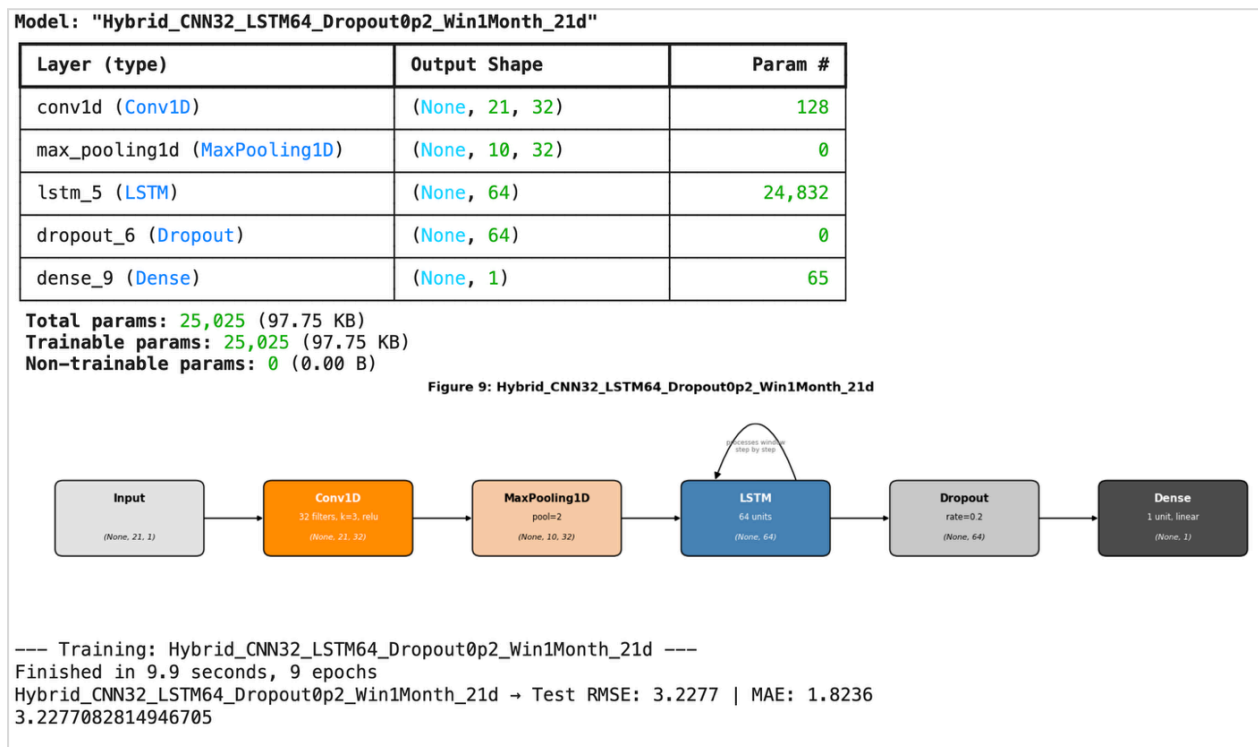


Figure 21. CNN-LSTM hybrid training result

Fine-tuning with learning rate: 0.00010000000474974513

--- Training: Hybrid_CNN32_LSTM64_Dropout0p2_Win1Month_21d_FineTuned ---
Finished in 10.1 seconds, 12 epochs
Hybrid_CNN32_LSTM64_Dropout0p2_Win1Month_21d_FineTuned → Test RMSE: 3.2361 | MAE: 1.8241
3.236129060937397

Figure 22. Fine-tuned hybrid training result

Overfitting control throughout relied on early stopping with patience 5 and restoration of best weights. Training times for the neural runs ranged from about 6.0 seconds to 45.7 seconds depending on architecture and window. Figures 23 and 24 show training and validation loss for the main models.

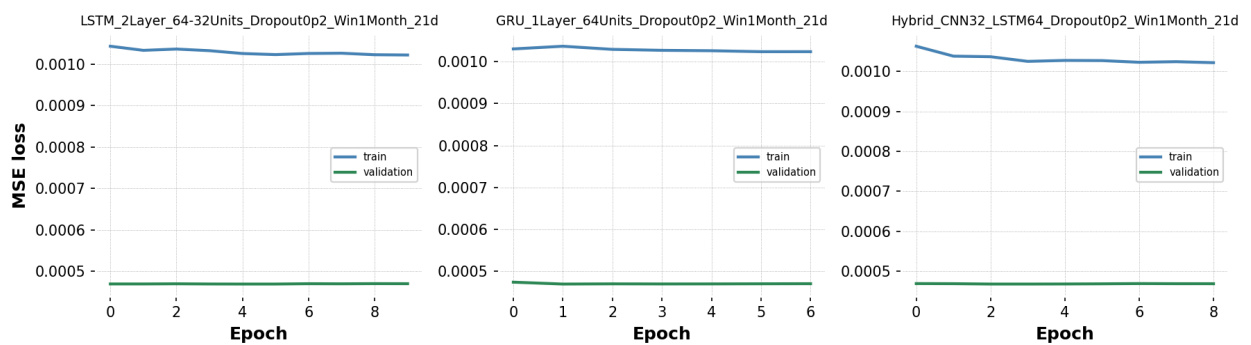


Figure 23. Training and validation loss for the main models (linear scale)

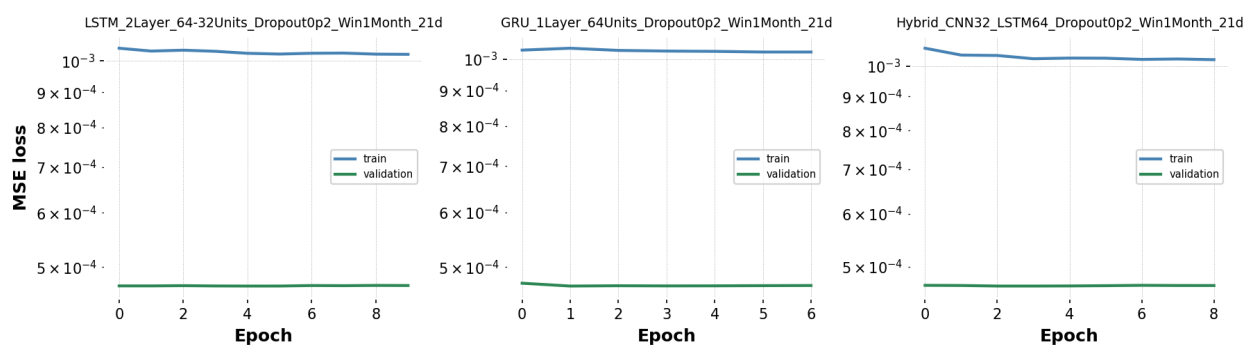


Figure 24. Training and validation loss for the main models (log scale)

6. Results and comparative analysis

6.1 Overall ranking

Table 1 collects all twelve evaluated configurations, sorted by test RMSE. The single most important row is the baseline. Persistence scores 3.2380 USD RMSE with MAPE 1.159%. Every neural model lands within a narrow band of that number. The clustering is not a coding accident; it reflects the near-unpredictability of one-step daily moves.

Six of the eleven neural configurations beat persistence on test RMSE. The CNN-LSTM hybrid leads at 3.2277. The lower-learning-rate GRU is second at 3.2339.

The best plain LSTM (two-layer, dropout) scores 3.2346. The best plain GRU with default learning rate and dropout scores 3.2383, essentially level with persistence.

Table 1. Test-set results for all models, sorted by RMSE

Model	Test RMSE	Test MAE	MAPE %	Parameters	Train (s)
CNN-LSTM hybrid	3.2277	1.8236	1.157	25,025	9.9
GRU, LR=0.0001, no dropout	3.2339	1.8245	1.159	12,929	6.6
LSTM 2-layer, dropout 0.2	3.2346	1.8255	1.159	29,345	13.3
LSTM 1-layer, dropout 0.2	3.2358	1.8255	1.159	16,961	9.6
CNN-LSTM fine-tuned	3.2361	1.8241	1.158	25,025	10.1
GRU 2-layer, dropout 0.2	3.2367	1.8249	1.159	22,305	9.5
Persistence baseline	3.2380	1.8269	1.159	0	0.0
GRU 1-layer, no dropout (1M)	3.2381	1.8249	1.158	12,929	7.6
GRU 1-layer, dropout 0.2	3.2383	1.8250	1.158	12,929	7.2
LSTM 2-layer, no dropout	3.2387	1.8267	1.160	29,345	7.3
GRU 1-layer, no dropout (1W)	3.2554	1.8463	1.177	12,929	6.0
GRU 1-layer, no dropout (1Y)	3.2762	1.8484	1.177	12,929	45.7

=== FINAL RESULTS (sorted by Test RMSE) ===

Model	Test RMSE	Test MAE	Test MAPE %	Parameters	Train time (s)
Hybrid_CNN32_LSTM64_Dropout0p2_Win1Month_21d	3.227708	1.823621	1.157150	25025	9.9
GRU_1Layer_64Units_NoDropout_LR0p0001_Win1Month_21d	3.233908	1.824519	1.159049	12929	6.6
LSTM_2Layer_64-32Units_Dropout0p2_Win1Month_21d	3.234587	1.825491	1.159343	29345	13.3
LSTM_1Layer_64Units_Dropout0p2_Win1Month_21d	3.235766	1.825475	1.159469	16961	9.6
Hybrid_CNN32_LSTM64_Dropout0p2_Win1Month_21d_FineTuned	3.236129	1.824074	1.158056	25025	10.1
GRU_2Layer_64-32Units_Dropout0p2_Win1Month_21d	3.236748	1.824946	1.158618	22305	9.5
Baseline (persistence)	3.238027	1.826946	1.158924	0	0.0
GRU_1Layer_64Units_NoDropout_Win1Month_21d	3.238113	1.824880	1.158121	12929	7.6
GRU_1Layer_64Units_Dropout0p2_Win1Month_21d	3.238285	1.825046	1.158350	12929	7.2
LSTM_2Layer_64-32Units_NoDropout_Win1Month_21d	3.238664	1.826687	1.159942	29345	7.3
GRU_1Layer_64Units_NoDropout_Win1Week_5d	3.255422	1.846295	1.177117	12929	6.0
GRU_1Layer_64Units_NoDropout_Win1Year_252d	3.276246	1.848362	1.176738	12929	45.7

Figure 25. Final Colab results table sorted by test RMSE

Because metrics cluster, Figures 26 to 30 show complementary views: zoomed accuracy, delta versus baseline, log-scaled cost, efficiency ratios and multi-criteria ranks.

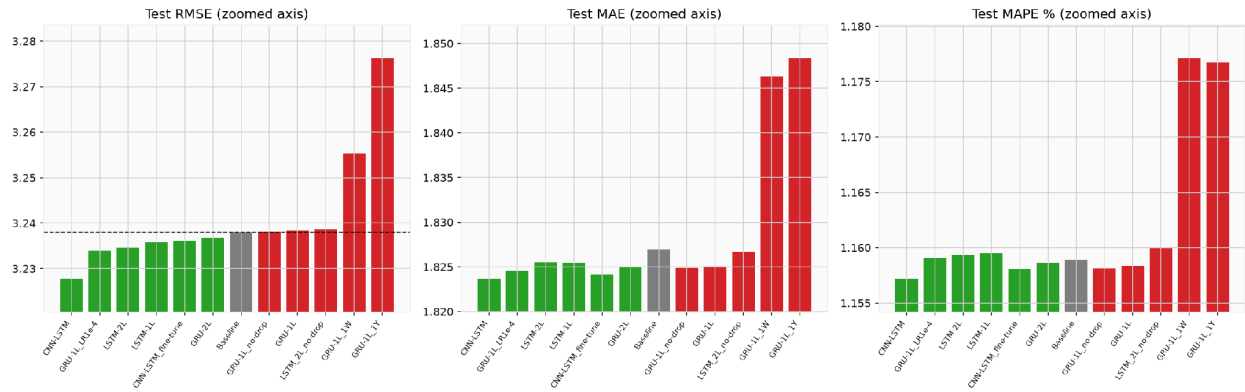


Figure 26. Zoomed test RMSE, MAE and MAPE by model

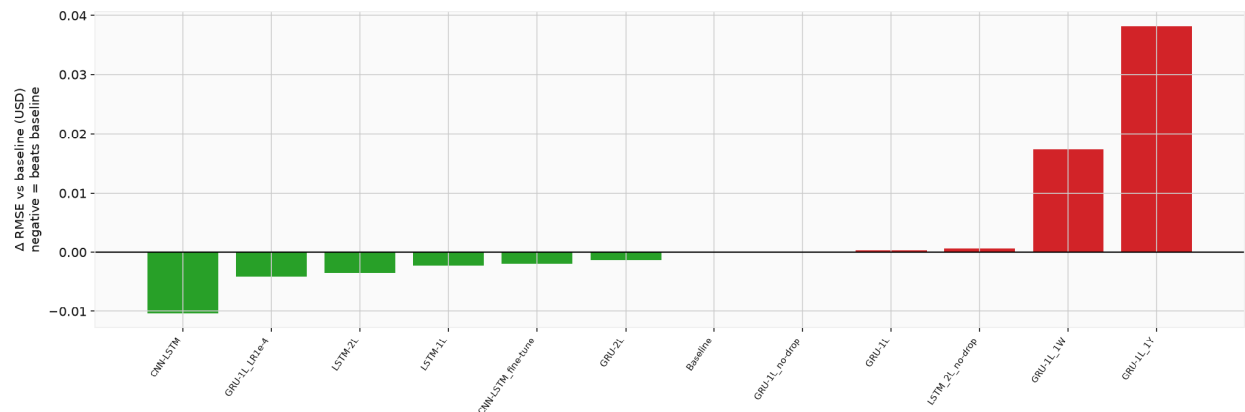


Figure 27. Change in test RMSE relative to the baseline

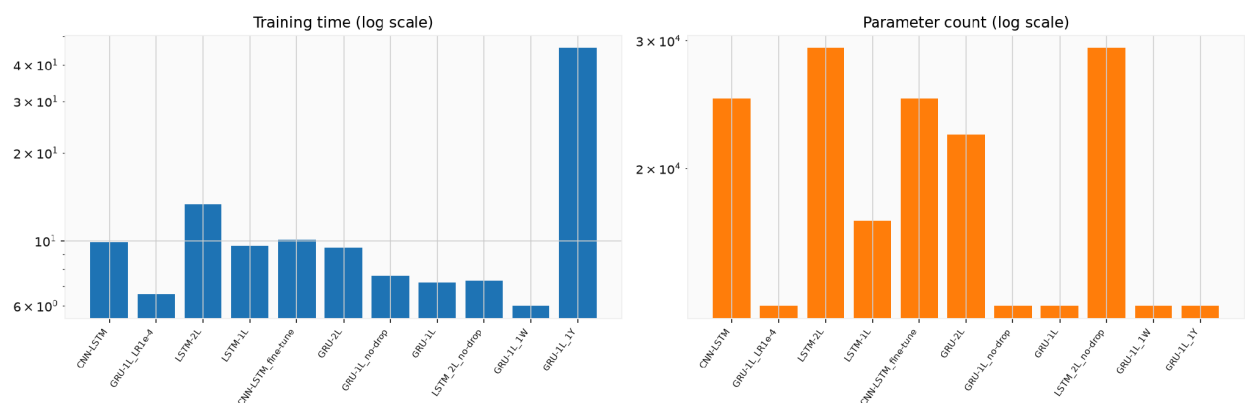


Figure 28. Training time and parameter count by model

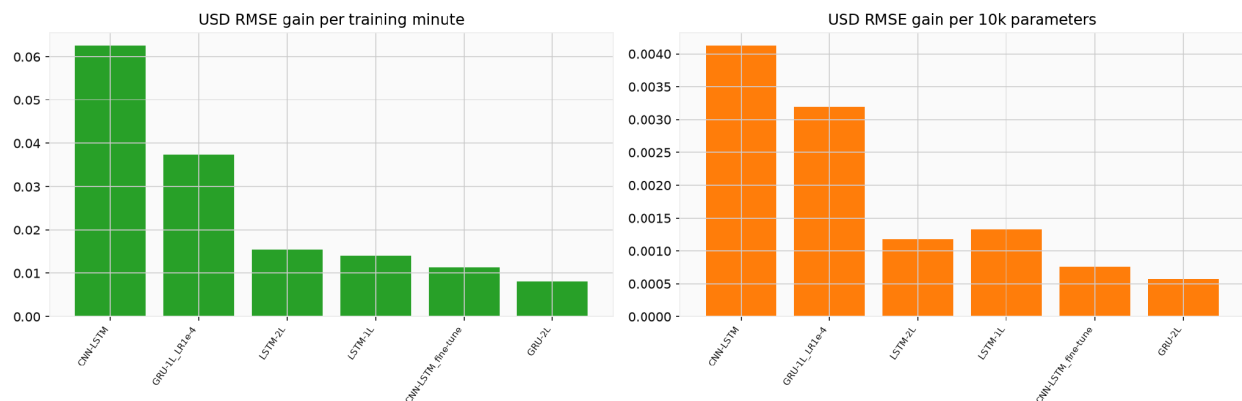


Figure 29. Efficiency of RMSE gain versus cost

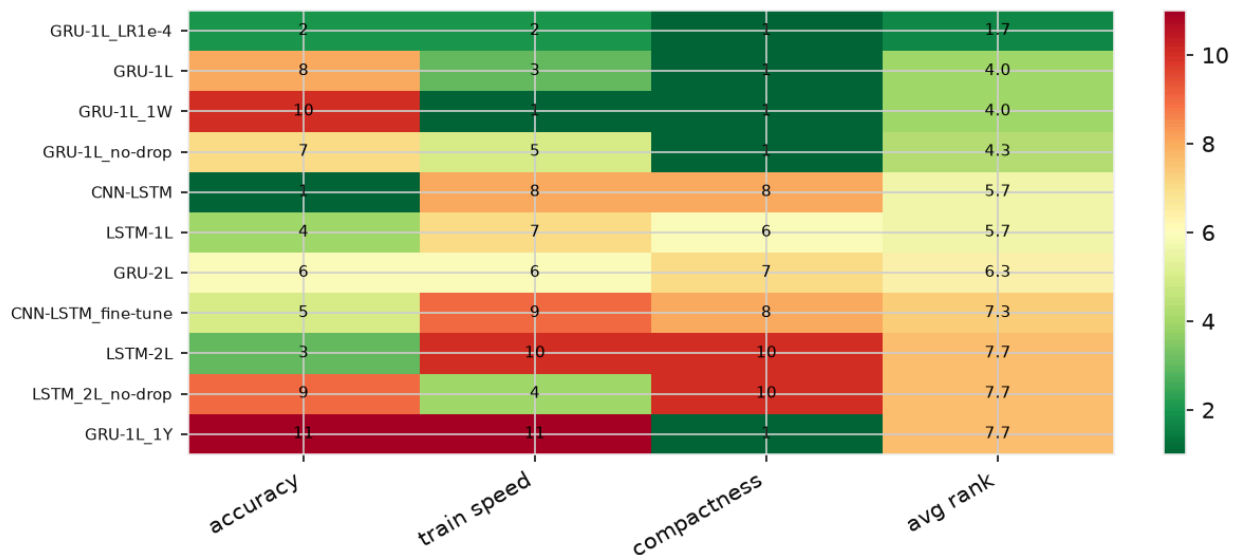


Figure 30. Multi-criteria rank heatmap of all models

6.2 LSTM versus GRU

On the comparison required by the brief, validation RMSEs are practically tied: best LSTM 0.8741, best GRU 0.8735, a 0.07% difference. The decisive tie-breaker is efficiency. At comparable depth the GRU uses fewer parameters and, for the one-layer dropout pair, trained in 7.2 seconds against 9.6 seconds for the LSTM. That accuracy-efficiency trade-off is consistent with applied stock-forecasting comparisons in which GRU often provides a strong balance relative to heavier recurrent variants (Sahu et al., 2025). My verdict is therefore that **GRU is the more suitable required architecture** for this scenario: similar accuracy at lower cost (Figure 31).

```

... === CONCLUSION ===
Best overall model : Hybrid_CNN32_LSTM64_Dropout0p2_Win1Month_21d (test RMSE 3.2277 USD)
Persistence baseline: 3.2380 USD
-> The best model beats the baseline by 0.32%. A small edge, and it should not be overstated.

LSTM vs GRU (validation RMSE):
best LSTM LSTM_2Layer_64-32Units_Dropout0p2_Win1Month_21d (0.8741),
best GRU GRU_1Layer_64Units_Dropout0p2_Win1Month_21d (0.8735), a 0.07% difference.
GRU uses fewer parameters, so it is the more efficient choice at similar accuracy.

CNN-LSTM extension: adding a convolution front-end to the LSTM,
improved on the plain LSTM under the same training protocol.

All numbers above come from this notebook's own results table. I did not copy any result from elsewhere.

```

Figure 31. Final notebook conclusion for hybrid and GRU verdict

6.3 What the prediction plot shows

Figure 32 plots predicted and actual Adjusted Close over the final test year. The curves appear to track the market, including the March 2020 crash. On closer inspection the forecasts lag actual prices by roughly one day. In other words, the networks have largely learned a slightly refined form of persistence. That visual reading aligns with the numeric clustering around the baseline and warns against over-interpreting a smooth chart as strong predictive power.

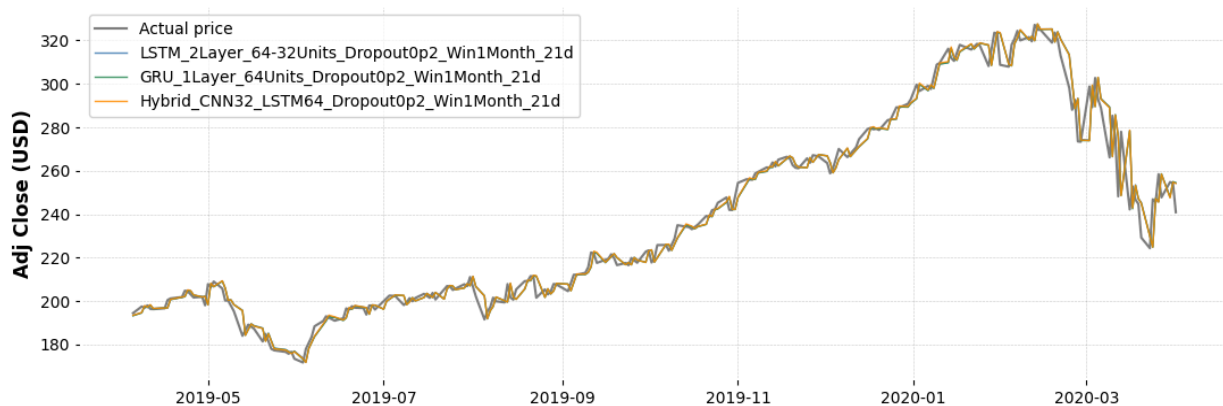


Figure 32. Predicted versus actual Adjusted Close (final test year)

6.4 Summary of Task 1 findings

The hybrid extension delivered the best accuracy and the clearest baseline-beating margin. Among the required architectures, GRU wins on efficiency at nearly identical validation accuracy. Improvements remain small enough that Section 7 must ask what the models are worth in use, and what ethical constraints follow from that reading.

7. Critical evaluation

7.1 Interpreting performance

Taken at face value, the best model beats persistence by about 0.32% of RMSE (3.2277 versus 3.2380 USD). Taken honestly, that summary needs qualifications. First, the margin is small enough that configuration choices can reorder models: fine-tuning actually hurt the hybrid on this run (3.2361 versus 3.2277). Second, every model, baseline included, sits near 1.16% MAPE, so none offers a step change in predictive power. Third, an earlier multi-seed check on a previous notebook version produced direction accuracy near 51-52%, no better than a trivial always-up rule. A model that tracks levels but cannot call tomorrow's direction has limited economic value for trading.

The correct inference is therefore not that deep learning predicts stock prices in a commercially decisive way. On this single series, one step ahead, deep learning mostly reproduces tomorrow-equals-today, and only the CNN-LSTM extracts a thin additional signal under a fixed seed.

7.2 Why the ceiling is low

Daily equity returns are close to serially uncorrelated, which is the empirical core of the efficient-market view of short-horizon prices (Fama, 1970). My models see only a short window of past prices for one stock. They do not observe earnings, news, interest rates or the pandemic shock that ends the test window. Given that information set, learning a near-persistence mapping may be the rational optimum. The experimental design was sound; the ceiling belongs largely to the problem. The poor 1Y ablation supports the same reading: adding a year of price history did not supply the missing fundamentals, it mainly added noise.

7.3 Business and investment implications

In the AI Analyst role I would not recommend automated buy or sell rules from this pipeline alone. Three practical messages follow. First, short-term level forecasts with test RMSE near 3.23 USD are too thin an edge for confident trading after transaction costs. Second, direction timing looks unreliable on the available supplementary evidence. Third, horizon choice matters: 1M is best and 1Y is worst among the tested windows, so simply using more history is not a strategy.

For leadership I would summarise as follows. The stock's long-run growth and sharp drawdowns are clear in the candlestick analysis, but the models do not provide a

dependable map of the next day. Any exploratory forecast should carry explicit error bands (for example plus or minus one test RMSE), require human sign-off, and state that a favourable back-test does not guarantee future profit. A safer internal use is a monitoring sandbox that flags when model error drifts relative to persistence, not an autonomous trading engine.

7.4 Reliability and limitations of the study

The strongest parts of the process are the chronological split, leakage-aware per-window normalisation, validation-only selection, a shared protocol across models, descriptive run names, and a reproducible public repository. The code was written in an explicit step-by-step style so that a marker or colleague can audit decisions without reverse-engineering abstractions.

Limitations are equally specific. The study uses a single train/validation/test split rather than walk-forward validation. The test window ends in the COVID crash, which is a demanding but also special regime. Volume is unused despite being present in the file. A multi-seed robustness notebook exists from an earlier code version and was not re-run on this exact notebook. These constraints bound how far absolute rankings should travel beyond this assessment.

7.5 Ethical implications

A stock predictor is not ethically neutral because people may act on it with real money (Ammanath, 2022; Gambelin, 2024). Overtrust is the first risk: Figure 32 looks compelling, yet the honest content beyond persistence is small. Automation bias and blurred accountability follow if forecasts are treated as advice without a named owner who can explain errors (Bartneck et al., 2021). In wider deployment there is also a fairness concern if sophisticated users act faster than retail clients on weak signals that appear stronger than they are.

Mitigations consistent with responsible practice, and used in this study's framing, are: always report against a naive baseline; communicate uncertainty in plain language; keep a human in the loop for decisions with financial impact; and never present this research artefact as investment advice.

7.6 Position relative to developments in the field

The GRU efficiency result aligns with applied comparisons in which GRU often matches or approaches LSTM accuracy at lower parameter cost on financial series (Sahu et al., 2025). The hybrid result is consistent with literature arguing that

convolution-plus-recurrence, sometimes with attention or ensemble framing, can help when local structure precedes longer memory (Bhanujyothi and Jacob, 2025; Zhao et al., 2025; Rokde et al., 2025). My contribution is not a claim of market-beating alpha. It is a controlled, reproducible demonstration that, under leakage-aware protocol and a hard baseline, those architectural stories appear only as small numeric edges on this dataset. From an AI-systems perspective, the exercise also illustrates a broader point in Russell and Norvig (2021): learning from data is powerful only to the extent that the available features and problem formulation contain usable signal.

8. Conclusion and reflection

This portfolio designed, trained and compared LSTM and GRU models for one-step-ahead Adjusted Close forecasting, extended the study with a CNN-LSTM hybrid, and evaluated every claim against persistence. The hybrid achieved the best test RMSE (3.2277). Among the architectures required by the brief, GRU is the more suitable choice because it matches LSTM validation accuracy within 0.07% while using fewer parameters and less training time. Window ablation favoured a one-month trading-day lookback over one week or one year. Fine-tuning did not improve the hybrid.

Keras and cloud notebook tooling removed most engineering friction. The difficult parts were experimental honesty: protecting time order, selecting only on validation data, and resisting a beautiful but lagging prediction chart. Starting with persistence reframed every later number and prevented tiny RMSE gaps from being mistaken for breakthroughs.

If the investigation were repeated, I would add walk-forward evaluation, include volume as a feature, and score direction or returns explicitly. What I take forward is calibrated scepticism: make leakage hard, freeze verified metrics before writing about them, and refuse to invent market insight that the data do not support. Deep learning remains a powerful tool, but it cannot conjure signal that short-horizon prices have already made difficult to extract.

Reference List

Ammanath, B., (2022). *Trustworthy AI: A Business Guide for Navigating Trust and Ethics in AI* [online]. Hoboken: Wiley.

Bartneck, C., Lütge, C., Wagner, A. and Welsh, S., (2021). *An Introduction to Ethics in Robotics and AI* [online]. Cham: Springer.

Bhanujyothi, H. C. and Jacob, I. J., (2025). A hybrid CNN-LSTM attention-based deep learning model for stock price prediction using technical indicators. *Engineering, Technology & Applied Science Research* [online], 15 (5), pp. 28012-28017.

Chollet, F. and Watson, M., (2025). *Deep Learning with Python* [online]. 3rd edition. Shelter Island: Manning.

Ekman, M., (2021). *Learning Deep Learning: Theory and Practice of Neural Networks, Computer Vision, NLP, and Transformers using TensorFlow* [online]. Boston: Addison-Wesley.

Fama, E. F., (1970). Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, 25 (2), pp. 383-417.

Gambelin, O., (2024). *Responsible AI: Implement an Ethical Approach in your Organization* [online]. London: Kogan Page.

Rokde, C., Chakole, J. and Ukey, A., (2025). Financial forecasting with deep learning models based ensemble technique in stock market analysis. *International Journal of Information Engineering and Electronic Business* [online], 17 (4), pp. 1-13.

Russell, S. J. and Norvig, P., (2021). *Artificial Intelligence: A Modern Approach* [online]. 4th edition. Harlow: Pearson.

Sahu, S. K., Mokahde, A. S., Chakole, J., Ajani, S. N. and Goswami, M. M., (2025). Performance analysis of LSTM, Bidirectional LSTM (BiLSTM), Gated Recurrent Unit (GRU), Convolutional LSTM (ConvLSTM) and LSTM with Attention in stock market prediction. *International Journal of Basic and Applied Sciences* [online], 14 (SI-2), pp. 131-140.

Zhao, G., Cheng, Y., Yang, J. and Ouyang, J., (2025). A dual-path convolutional neural network combined with an attention-based bidirectional long short-term memory network for stock price prediction. *PLoS ONE* [online], 20 (4), article e0319775.

Appendix A. Code repository and final artefacts

Source code is not pasted into this portfolio. The executable workflow and the frozen outputs used for Table 1 and Figures 1-32 are available in the public repository below, so a marker can open the final files directly.

Repository (public): https://github.com/kvmmn/COM7019_AI-NN

A.1 Primary executable notebook

Item	Path in repository	Direct link
Executed Jupyter notebook (final)	notebook/COM7019_25199053.ipynb	https://github.com/kvmmn/COM7019_AI-NN/blob/main/notebook/COM7019_25199053.ipynb

The notebook covers data loading and quality checks, exploratory candlestick charts, chronological splitting, windowing, the persistence baseline, LSTM and GRU training, dropout and window ablations, the learning-rate experiment, the CNN-LSTM hybrid, fine-tuning, comparative tables and figures, and the final numeric conclusion.

A.2 Authorised dataset

Item	Path in repository	Direct link
Stock price CSV used in all experiments	data/Stock_Price_Data_[3921].csv	https://github.com/kvmmn/COM7019_AI-NN/blob/main/data/Stock_Price_Data_%5B3921%5D.csv

A.3 Results tables (frozen run)

Item	Path in repository	Direct link
Full run comparison (source for Table 1)	results/tables/results_all_runs.csv	https://github.com/kvmmn/COM7019_AI-NN/blob/main/results/tables/results_all_runs.csv
Enriched comparison table	results/tables/results_comparison_enriched.csv	https://github.com/kvmmn/COM7019_AI-NN/blob/main/results/tables/results_comparison_enriched.csv
Model architecture text summaries	results/models/model_summaries.txt	https://github.com/kvmmn/COM7019_AI-NN/blob/main/results/models/model_summaries.txt